



Safety Lab

Deployment and Setup Guide

Run Safety Lab Aero on your own infrastructure: your server, your desktop, your AI key. Your data never touches Safety Lab.

safetylabaero.com

Document No.	SL-DG-0001
Title	Safety Lab Aero: Deployment and Setup Guide
Revision	2.0
Date	14 September 2026
Prepared by	Safety Lab Aero
Status	Draft

Revision history

Rev	Date	Summary of changes
1.0	7 Sep 2026	Initial release. Self-hosted backend (database, AI proxy, application configuration), defense / ITAR GovCloud variant, browser-only mode, and the isolation checklist.
2.0	14 Sep 2026	Adds the desktop application (install, first launch, pointing it at your server and AI, files-only mode, updates), the two ways to hold an AI key and how to create one, a data-residency overview of the three ways to run the product, and team setup. Restructured into a numbered hierarchy (sections 4 to 7 with subsections).

Contents

1 What this sets up.....	4
2 Where your data lives.....	4
3 What you need first.....	4
4 Your server.....	5
4.1 Stand up the database.....	5
4.2 Deploy the AI proxy.....	5
4.3 Host the web application (optional for desktop-only teams).....	6
4.4 Sign in and set up your team.....	6
5 The desktop application.....	8
5.1 Install it.....	8
5.2 First launch.....	8
5.3 Point the desktop at your server and your AI.....	8
5.4 Files-only alternative.....	9
5.5 Updates.....	9
6 AI keys.....	10
6.1 How AI reaches your model.....	10
6.2 Creating your AI key.....	10
6.3 Per-user keys ("bring your own key").....	10
6.4 Defense and ITAR (GovCloud).....	11
7 Verify and finish.....	12
7.1 Automatic invite emails (optional).....	12
7.2 Verify isolation.....	12
7.3 What "done" looks like.....	12
7.4 Getting help.....	12

1 What this sets up

When you finish, your project data, your sign-ins, and your AI all live on infrastructure you control. Safety Lab holds nothing and is never in the data path.

Safety Lab Aero is delivered as files you host yourself: the application, a small database, a lightweight AI proxy that holds your AI key and checks your license, and a desktop application for each user. This guide walks through standing all of them up on your own infrastructure. Nothing in these steps sends data to Safety Lab, and the application refuses to start if any setting still points back at us. That refusal is deliberate: it is the guardrail that keeps your certification data on your side.

Revision 2.0 adds the desktop application, the two ways to hold an AI key, and a plain description of where every piece of data lives. If you set up with revision 1.0, nothing you did changes; sections 5 and 6 are the new material.

2 Where your data lives

There are three ways to run the product. Most teams use the first, with the desktop application on each user's machine pointed at it.

Way to run it	Where projects live	Sign-in	AI	Best for
Your own server	Your own database (a Supabase project in your organization, or your own PostgreSQL)	Your database's sign-in, your accounts	Your AI proxy, holding your key	A team: shared projects, reviewers, version history
Desktop, files only	Project files on the user's machine, or your SharePoint / OneDrive	None needed; the license covers the machine	Your AI proxy, or off	A single engineer, or an air-gapped machine
Browser only	The user's browser storage on that machine	None	Off unless the user supplies a key	Evaluation on one machine, no install

In every case the answer to "what does Safety Lab hold" is the same: nothing. No project, no document, no account, no sign-in event, and no AI request passes through or is stored on a Safety Lab system. The application's network guard enforces this: the desktop application can only reach the addresses you configure, and the web application refuses to start if any address points at Safety Lab.

The desktop application and the web application are two views of the same thing. A team on its own server can use either or both: sign in once, see the same projects, edit in either, and open a project from the web in the desktop or the other way around.

3 What you need first

- **A database you control.** The simplest is a Supabase project in your own organization; it also provides the sign-in and the live updates the application uses. A self-managed PostgreSQL database also works.

- **One AI option.** Your own Anthropic (Claude) API key, or Azure OpenAI, or for defense programs AWS Bedrock GovCloud or Azure OpenAI Government (section 6.4).
- **Somewhere to run the AI proxy.** A Cloudflare account, or any host that runs Node 18 or later: a container or virtual machine in your own cloud.
- **The delivered files.** The application files, the AI proxy files, the database install kit, and the desktop installers for your users' machines. Safety Lab provides all of them.
- **Your license file and public key.** Your license (a small file ending in .lic) and the public-key list used to check it. The public key is safe to share and is not a secret.

If your users only ever work in the desktop application, you can skip hosting the web application (section 4.3); everything else in section 4 still applies.

4 Your server

4.1 Stand up the database

From the database folder of the delivered kit, run the installer against your database connection string. In Supabase this is under Project Settings, Database, Connection string.

```
./apply.sh "postgresql://postgres:PASSWORD@db.YOURREF.supabase.co:5432/postgres"
```

Then make yourself an administrator (run in the Supabase SQL editor, or with psql):

```
insert into private.platform_admins(email) values ('you@yourcompany.com');
```

Your database is ready. The install starts with an empty administrator list, you just added the first entry, and carries none of Safety Lab's own settings. To confirm the database is healthy without opening the application, run the checker from the kit against the same connection string:

```
./verify.sh "postgresql://postgres:PASSWORD@db.YOURREF.supabase.co:5432/postgres"
```

4.2 Deploy the AI proxy

The proxy is the piece that holds your AI key. Every AI request from the application goes to the proxy, the proxy checks your signed license, and only then forwards the request to your AI provider using your key. No user's machine ever holds the key, and your provider bills you directly for your own usage. Both ways of running it use the same three settings:

- **LICENSE_MODE** set to offline.
- **LICENSE_PUBLIC_KEYS**, the public-key list Safety Lab gave you (a short JSON array).
- **ANTHROPIC_API_KEY**, your own Claude key (section 6.2 explains how to create one). For Azure OpenAI, use the Azure settings in the proxy folder instead.

Option A. On Cloudflare

From the proxy folder:

```
npx wrangler secret put LICENSE_PUBLIC_KEYS
npx wrangler secret put ANTHROPIC_API_KEY
npx wrangler deploy
```

Set `LICENSE_MODE=offline` as a plain variable in the proxy configuration, then note the address Cloudflare gives you (for example `https://your-proxy.workers.dev`).

Option B. As a standalone service in your own cloud

Anywhere that runs Node 18 or later, a container or virtual machine inside your own network:

```
cp .env.example .env      # set LICENSE_MODE, LICENSE_PUBLIC_KEYS, ANTHROPIC_API_KEY
node server.mjs           # listens on :8787
```

Put it behind your normal HTTPS front door and note that address. Confirm it is alive: a request to `/v1/ai/health` returns `{"ok":true}`. Write the proxy address down; you will enter it in the web configuration (section 4.3) and in every desktop application's settings (section 5.3).

4.3 Host the web application (optional for desktop-only teams)

Serve the application files from your own web host. The application needs a short configuration script before its own scripts. The easiest way is to generate it: copy `install.env.example` to `install.env` in the delivered kit, fill in your addresses, and run the generator.

```
cp install.env.example install.env  # then edit install.env
node configure.js                   # writes slab_env.js
```

Include the `slab_env.js` it produces before the application's own scripts. The generator refuses to write a file that points anything at a Safety Lab address, and it catches the one easy mistake, leaving the AI endpoint blank, which would otherwise be an error at startup. If you prefer to write it by hand, add this instead, filling in your addresses:

```
<script>
  window.__SLAB_SUPABASE_URL__ = "https://YOURREF.supabase.co";
  window.__SLAB_SUPABASE_KEY__ = "YOUR_SUPABASE_ANON_KEY"; // publishable key
  window.__SLAB_AI_ENDPOINT__ = "https://your-proxy-address";
  window.__SLAB_WEB_APP_URL__ = "https://app.yourcompany.com"; // this install
</script>
```

If any of these still points at a Safety Lab address, the application refuses to start and names the setting.

4.4 Sign in and set up your team

Open the application (web or desktop). On first run it asks for your license file; load it. Create your account and sign in. Then:

1. **Create a workspace.** Whoever creates it is its owner. Nobody joins a workspace without an invitation.
2. **Invite your team.** From the workspace, invite each member by email. Without the optional invite-email function (section 7.1) the application shows an invitation link for you to pass along by hand; with it, the invitation is emailed automatically.
3. **Set roles.** Owners and administrators manage membership; editors work on projects; viewers are read-only reviewers. Only the owner can transfer ownership, and administrators cannot remove the owner.

4. **Make a test project** and confirm it appears in your own database tables, not ours, and that AI drafting works, going through your proxy to your AI.

5 The desktop application

5.1 Install it

The desktop application is the same product in a window of its own: it opens project files, works offline, and syncs with your server when it is back online. Safety Lab provides installers for each platform.

macOS

Two installers are provided, one for Apple silicon (arm64) and one for Intel (x64). Open the .dmg and drag Safety Lab Aero to Applications. On the first launch, right-click the application and choose Open, then confirm. macOS shows a warning the first time because the current build is not yet notarized with Apple; the right-click step is only needed once. Later builds will be notarized, at which point the warning goes away.

Windows

Run the installer provided by Safety Lab and follow the prompts. Windows may show a SmartScreen notice the first time; choose More info, then Run anyway. As with macOS, this is because the build is not yet signed with a publisher certificate, and it is a one-time step.

What the application can reach

The desktop application is fenced at the network layer. It can only contact the addresses you enter in its settings: your server and your AI proxy. It cannot reach Safety Lab, fonts, analytics, or anything else, even if a script tried. There is nothing to configure for this; it follows from your settings.

5.2 First launch

The first time the application opens it walks through four short screens before the main window appears:

1. **License.** Choose your .lic file, or paste its contents. It is checked on this computer and never sent anywhere. Trials use a temporary license with an end date; the screen shows what the license covers and when it ends.
2. **End User License Agreement.** Read and accept.
3. **Software License and Subscription Agreement.** Read and accept.
4. **Screen lock (optional).** A passcode adds a lock screen when the application opens. It is a screen lock, not an identity: you sign in to your account inside the application exactly as on the web, and stay signed in on this computer. Leave it blank for no lock.

After this the application opens against the Safety Lab trial cloud by default. For a team on its own server, the next step is to point it at your server.

5.3 Point the desktop at your server and your AI

Open Settings: on macOS it is under the Safety Lab Aero menu (or Cmd+comma); on Windows it is under File (or Ctrl+comma). Settings are stored on that computer only.

Where your data lives

Choose **My organization's server** and enter three things:

- **Server address.** Your database's address, for example `https://YOURREF.supabase.co`.
- **Server key (publishable).** The publishable, or anon, key from your Supabase project. This key is designed to be distributed to clients; it is not the secret service key.
- **Web address.** Only if you host the web application (section 4.3): its address, so "Open this project on the web" works. Leave blank otherwise.

AI

Choose **My organization's AI endpoint** and enter your proxy address from section 4.2, followed by `/v1/ai`, for example `https://your-proxy.workers.dev/v1/ai`. If your team does not use AI features, choose Off; everything else works.

Click Save and reload. The application checks the settings before it restarts and refuses, in plain words, if anything still points at Safety Lab or if a required address is missing. Then sign in with your account. You stay signed in on this computer.

Confirm it

- Open a project you created on the web. It should appear in the desktop, and an edit made in one should appear in the other.
- Run an AI draft. It goes through your proxy to your AI; nothing reaches Safety Lab.
- The License card in Settings shows your organization, tier, seats, and end date.

5.4 Files-only alternative

A single engineer, or a machine that must not talk to any server, can run the desktop application with no backend at all. In Settings choose **Files only**. Projects are then files you open and save from the File menu (Open Project File, Save Project File, Save As). Keep them anywhere you like, including a folder synced to your own SharePoint or OneDrive. AI can still run through your organization's proxy if you enter its address, or be off. There is no sign-in in this mode; the license covers the machine.

5.5 Updates

Updates are never installed silently. Use Help, Check for Updates (on macOS also under the Safety Lab Aero menu). If a newer version exists the application tells you and opens the download page; you install it the same way as the first time. Every update announcement is signed by Safety Lab and the application verifies that signature before believing it, so a tampered download page cannot push you a build we did not publish.

6 AI keys

6.1 How AI reaches your model

Every AI request from the application, web or desktop, goes to your proxy. The proxy holds your provider key, checks your signed license, and forwards the request to your AI provider. Three things follow from this design.

- **No user's machine holds the key.** Users see only the proxy address. Rotating the key is a change in one place.
- **Your provider bills you.** Usage runs on your own account with your provider. Safety Lab does not meter, cap, or see it.
- **Controlled work stays inside your boundary.** Projects marked export-controlled are only sent to the sovereign backend you configure (section 6.4); if none is configured the request is refused, never sent to a public model.

The AI never trains on your content, on any path. This is a contractual commitment as well as a technical one.

6.2 Creating your AI key

Anthropic (Claude)

1. Sign in to the Anthropic Console (console.anthropic.com) with an account that belongs to your organization, not an individual's personal account.
2. Under API Keys, create a key. Name it for this purpose, for example "Safety Lab Aero proxy", so it can be rotated independently later.
3. Set a monthly spending limit on the organization. The proxy has no cap of its own, so this is your budget control.
4. Copy the key once, into the proxy configuration (section 4.2) and nowhere else. Anthropic does not show it again.

Azure OpenAI

Create an Azure OpenAI resource in your subscription, deploy a model, and note the endpoint, deployment name, and key. Enter them in the proxy's Azure settings (the proxy folder documents the variable names). For defense programs use the Azure Government variant in section 6.4.

6.3 Per-user keys ("bring your own key")

The application also has an Advanced setting where a user can paste an Anthropic key of their own, which sends requests directly from that machine to Anthropic. It is useful on the Safety Lab trial cloud, where a user evaluating the product wants to use their own key without any setup.

On your own server, and in the desktop application, this path is not used, and that is by design. The desktop application can only reach the addresses you configured (section 5.1), and the web application on your server routes AI through your proxy. Both mean a personal key pasted into the application will not reach Anthropic from those installs. For a team we recommend the proxy for exactly this reason: one key, held in one place,

with the license check and the export-control fence in front of it. Users who evaluated the product on the trial cloud with a personal key can remove it under Advanced AI settings once they move to your server.

6.4 Defense and ITAR (GovCloud)

Same as section 4, but run the proxy as the standalone service (Option B in section 4.2) inside your own AWS GovCloud network, and point it at a US-sovereign AI instead of a public key. In the proxy's environment set one of the following.

AWS Bedrock GovCloud (Claude, the engine the product was validated on)

```
ITAR_PROVIDER=bedrock
AWS_BEDROCK_REGION=us-gov-west-1
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_BEDROCK_MODEL_ID=anthropic.claude-3-5-sonnet-20240620-v1:0
```

Azure OpenAI Government

Set `ITAR_PROVIDER=azure` and the Azure variables (see the ITAR provisioning notes in the proxy folder).

If no sovereign backend is configured, a controlled request is refused, never sent to a public model. Mark controlled projects as export-controlled in the application; the data fence and this proxy both enforce it.

7 Verify and finish

7.1 Automatic invite emails (optional)

Not required. Without it, when an administrator invites a teammate the application shows a link to pass along by hand. If you want invitations emailed automatically, deploy the one invite-email function to your database with your own email-provider key. It is the only server-side function a self-hosted install can use, and it is optional.

7.2 Verify isolation

In the browser console of the web application, or in the desktop application's View menu with developer tools enabled by Safety Lab, run **SLConfigEgress()**. It lists every address the application will contact. Every one should be yours: your database, your proxy. If anything shows a Safety Lab address, correct the matching setting. On the database side, `verify.sh` (section 4.1) checks the tables, security rules, and administrator account are in place and prints a pass or fail summary.

7.3 What "done" looks like

- Your database holds your projects; ours holds nothing.
- Your proxy holds your AI key; no user's machine ever sees it; your provider bills you; controlled work never leaves your boundary.
- Each desktop application is pointed at your server and your proxy, and can reach nothing else.
- `SLConfigEgress()` lists only your own addresses.
- Your team is invited, roles are set, and a test project round-trips between web and desktop.

7.4 Getting help

Your onboarding session with Safety Lab covers this guide end to end on your own infrastructure, remote or on site. For anything after that, contact your Safety Lab representative; support is not capped.